P4air: Comprehensive Implementation Details

(VM-Only — BMv2 + Mininet Environment)

> **Paper**: *P4air: Increasing Fairness among Competing Congestion Control Algorithms*
> **Authors**: Belma Turkovic & Fernando Kuipers (TU Delft)
> **Purpose**: Baseline implementation for the **KBCS** project

> [!IMPORTANT]
> This entire implementation is designed for a **software-only environment**:
> **P4 VM** with `simple_switch` (BMv2), **Mininet**, **Ryu**, and standard Linux tools.
> No hardware switches (Tofino/Netronome), no IoT devices, no physical testbed required.
> All experiments are emulated using Mininet with configurable link parameters.

Table of Contents

1. Paper Summary
2. Architecture Overview
3. Phase 1 — Environment & Scaffolding
4. Phase 2 — P4 Headers, Parser & Basic Pipeline
5. Phase 3 — Fingerprinting Module
6. Phase 4 — Reallocation Module
7. Phase 5 — Apply Actions Module
8. Phase 6 — Evaluation & Comparison
9. Key Equations
10. Parameter Reference

---

1. Paper Summary

Problem

Different congestion control algorithms (CCAs) co-existing on a shared bottleneck leads to **unfair bandwidth distribution** — aggressive flows (Cubic, Reno) overpower conservative ones (Vegas, BBR).

Solution — P4air

A P4 data-plane application that:

1. **Fingerprints** flows into 4 CCA groups based on behavior
2. **Reallocates** queues dynamically among groups
3. **Applies custom actions** per group to enforce fairness

The 4 Congestion Control Groups

Group	Metric	Examples	Aggressiveness
Purely Loss-Based	Packet loss	Cubic, Reno, BIC	Highest
Loss-Delay	Loss + delay	Illinois, YeAH, Veno, Hybla	Medium
Delay-Based	RTT increase	Vegas, LoLa	Lowest
Model-Based	BDP model	BBR	Variable (periodic)

Key Results from Paper

- Fairness: **~0.87–0.96** (vs ~0.18–0.49 without P4air)
- Utilization: **≥90%** maintained
- Runs on both BMv2 (software) and Tofino (hardware)

---

2. Architecture Overview

```
┌──────────────────────────────────────────────────────────────────┐
│              P4air Pipeline (BMv2 simple_switch)                 │
│                                                                  │
│   ┌──────────────── INGRESS ──────────────────┐                  │
│   │                                           │                  │
│   │  ① FINGERPRINTING (ingress part)          │                  │
│   │     • Hash 5-tuple → flow_id              │                  │
│   │     • Track num_pkts per RTT interval     │                  │
│   │     • RTT estimation (3-way handshake)    │                  │
│   │     • Detect slow-start end               │                  │
│   │     • Update BwEst counter                │                  │
```

```
│   │     • Read/update group from register   │                │
│   │                                          │                │
│   │  ② REALLOCATION (ingress part)           │                │
│   │     • Process recirculated packets       │                │
│   │     • Update group boundaries (li)       │                │
│   │     • Assign queue based on group        │                │
│   │                                          │                │
│   │  ③ APPLY ACTIONS                         │                │
│   │     • delay-based  → recirculate (delay)│                │
│   │     • loss/loss-delay → drop packet      │                │
│   │     • model-based → modify TCP window    │                │
│   └──────────────────────────────────────────┘                │
│                        ↓                                      │
│   ┌──── QUEUING (simple_switch priority queues) ────┐         │
│   │  Q0: Ants │ Q1: Mice │ Q2-Q7: 4 groups (RR)    │         │
│   └──────────────────────────────────────────────────┘         │
│                        ↓                                      │
│   ┌──────────────── EGRESS ──────────────────────┐             │
│   │                                              │             │
│   │  ④ FINGERPRINTING (egress part)              │             │
│   │     • Track enqueue depth                    │             │
│   │     • Update aggressiveness metric           │             │
│   │     • Recalculate group if patterns          │             │
│   │       detected                               │             │
│   │                                              │             │
│   │  ⑤ REALLOCATION (egress part)                │             │
│   │     • If group changed → recirculate         │             │
│   │       packet to update ingress register      │             │
│   └──────────────────────────────────────────────┘             │
└────────────────────────────────────────────────────────────────┘
```

Phase 1 — Environment & Scaffolding

1.1 Your VM Already Has

Based on your existing KBCS setup, you already have:

- ✅ `simple_switch` (BMv2) with `--priority-queues` support
- ✅ `p4c-bm2-ss` compiler (v1model architecture)
- ✅ Mininet with `TCLink` for bandwidth/delay shaping

- ✅ `iperf3`, `tcpdump`, `sysctl` for CCA switching
- ✅ Python 3, `simple_switch_CLI` for table management
- ✅ `P4Switch`, `P4Host` Mininet abstractions (from `utils/p4_mininet.py`)

1.2 Additional Packages to Install (in the P4 VM)

```
# All of these are lightweight, run inside your P4 VM
sudo apt install -y iperf3 tcpdump
pip3 install scapy matplotlib numpy pandas
```

1.3 Congestion Control Modules in VM

The paper tests 14 CCAs from the Linux kernel. On your VM, ensure these are available:

```
# Check available CCAs
sysctl net.ipv4.tcp_available_congestion_control

# Load modules needed for testing
sudo modprobe tcp_vegas
sudo modprobe tcp_bbr
sudo modprobe tcp_illinois
sudo modprobe tcp_veno
sudo modprobe tcp_yeah
sudo modprobe tcp_hybla
sudo modprobe tcp_westwood
sudo modprobe tcp_htcp
sudo modprobe tcp_bic
sudo modprobe tcp_highspeed

# Verify
cat /proc/sys/net/ipv4/tcp_available_congestion_control
```

> [!NOTE]
> Not all 14 CCAs may be available. **At minimum you need 4** (one per group):
> **Cubic** (loss), **Illinois** (loss-delay), **Vegas** (delay), **BBR** (model).
> Your KBCS topology already uses Cubic and BBR.

1.4 Directory Structure

```
Baseline/
├── P4air_Baseline_Paper.pdf
├── P4air_Implementation_Details.md     ← this file
│
├── p4air/
│   ├── p4src/                          # P4 source code
│   │   ├── p4air.p4                    # Top-level (like kbcs.p4)
│   │   ├── headers.p4                  # Headers + metadata + registers
│   │   ├── parser.p4                   # Parser & deparser
│   │   ├── ingress.p4                  # Ingress control
│   │   └── egress.p4                   # Egress control
│   │
│   ├── utils/                          # Reuse from KBCS
│   │   └── p4_mininet.py              # (symlink or copy from
kbcs/utils/)
│   │
│   ├── topology.py                     # Mininet topology (based on kbcs)
│   ├── runtime.json                    # Forwarding rules
│   ├── Makefile                        # Build & run
│   │
│   ├── experiments/
│   │   ├── run_fingerprinting_test.py  # Test CCA classification accuracy
│   │   ├── run_fairness_test.py        # Inter/intra fairness
│   │   └── run_comparison.py           # All baselines vs P4air
│   │
│   ├── analysis/
│   │   ├── calculate_fairness.py       # Jain's Index calculator
│   │   └── plot_results.py             # Generate comparison charts
│   │
│   ├── results/                        # (auto-generated)
│   └── logs/                           # (auto-generated)
│
├── baselines/
│   ├── no_aqm/                         # Simple forwarding (copy from
kbcs_baseline)
│   └── diff_queues/                    # Hash-based queue separation
```

1.5 Tasks — Phase 1

- [ ] Verify VM has `simple_switch`, `p4c-bm2-ss`, Mininet
- [ ] Install additional packages (`scapy`, `matplotlib`, etc.)

- [ ] Load and verify Linux CCA modules (at least Cubic, Vegas, Illinois, BBR)
- [ ] Create the directory structure
- [ ] Copy `utils/p4_mininet.py` from existing KBCS project
- [ ] Create skeleton Makefile modeled on KBCS's Makefile

Phase 2 — P4 Headers, Parser & Basic Pipeline

2.1 Headers (`headers.p4`)

> [!TIP]
> Reuse the same Ethernet/IPv4/TCP headers from your KBCS `headers.p4` .
>
> Only the **metadata struct** and **registers** change significantly.

Metadata struct — replaces KBCS's `local_metadata_t` :

```
struct p4air_metadata_t {
    // Flow identification
    bit<16> flow_id;                // Hash of 5-tuple

    // Classification
    bit<3>  flow_group;             // 0=ant, 1=mice, 2=delay, 3=loss-del
                                    // 4=purely-loss, 5=model-based
    bit<3>  prev_group;             // Previous group
    bit<1>  group_changed;          // Flag: reclassification occurred
    bit<1>  is_recirculated;        // Flag: this is a recirculated packe

    // RTT tracking
    bit<48> rtt_estimate;           // Estimated RTT (microseconds)
    bit<48> rtt_start;              // Start of current RTT interval

    // Per-RTT statistics
    bit<32> num_pkts;               // Packets in current RTT
    bit<32> num_pkts_prev;          // Packets in previous RTT
    bit<32> max_enq_len;            // Max enqueue depth (current RTT)
    bit<32> max_enq_len_prev;       // Max enqueue depth (previous RTT)

    // Fingerprinting metrics
    bit<8>  aggressiveness;         // Queue-fill rate tracker
    bit<8>  aggr_streak;            // Consecutive RTTs with growing queu
    bit<8>  bwest_counter;          // BW estimation pattern counter
```

```
    // Queue assignment
    bit<3>  assigned_queue;          // Queue number (0-7)
    bit<32> bdp;                     // Bandwidth-delay product
}
```

Register arrays:

```
#define FLOW_TABLE_SIZE  1024     // Max concurrent flows
#define NUM_GROUPS       6        // ant, mice, delay, loss-delay, loss,
#define NUM_QUEUES       8        // BMv2 --priority-queues 8

// Per-flow state
register<bit<3>>(FLOW_TABLE_SIZE)   reg_group;           // Current group
register<bit<3>>(FLOW_TABLE_SIZE)   reg_queue;           // Assigned queu
register<bit<48>>(FLOW_TABLE_SIZE)  reg_rtt;             // Estimated RTT
register<bit<48>>(FLOW_TABLE_SIZE)  reg_rtt_start;       // RTT interval
register<bit<32>>(FLOW_TABLE_SIZE)  reg_num_pkts;        // Pkts current
register<bit<32>>(FLOW_TABLE_SIZE)  reg_num_pkts_prev;   // Pkts previous
register<bit<32>>(FLOW_TABLE_SIZE)  reg_max_enq;         // Max enqueue c
register<bit<32>>(FLOW_TABLE_SIZE)  reg_max_enq_prev;    // Max enqueue p
register<bit<8>>(FLOW_TABLE_SIZE)   reg_aggr;            // Aggressivenes
register<bit<8>>(FLOW_TABLE_SIZE)   reg_aggr_streak;     // Consecutive a
register<bit<8>>(FLOW_TABLE_SIZE)   reg_bwest;           // BwEst counter
register<bit<1>>(FLOW_TABLE_SIZE)   reg_syn_seen;        // SYN flag seer
register<bit<48>>(FLOW_TABLE_SIZE)  reg_syn_ts;          // SYN timestamp

// Per-group state
register<bit<32>>(NUM_GROUPS)       reg_group_flows;     // Flows per gro
register<bit<3>>(NUM_GROUPS)        reg_group_q_start;   // Queue boundar
register<bit<3>>(NUM_GROUPS)        reg_group_q_end;     // Queue boundar
register<bit<32>>(NUM_GROUPS)       reg_group_seq_idx;   // Sequential ir

// Global
register<bit<32>>(1)                reg_total_flows;     // Total long fl
```

2.2 Parser & Deparser (`parser.p4`)

> Same structure as KBCS parser. Parse: Ethernet → IPv4 → TCP.

2.3 Top-Level (`p4air.p4`)

```
#include <core.p4>
#include <v1model.p4>
#include "headers.p4"
#include "parser.p4"
#include "ingress.p4"
#include "egress.p4"

V1Switch(
    P4airParser(),
    P4airVerifyChecksum(),
    P4airIngress(),
    P4airEgress(),
    P4airComputeChecksum(),
    P4airDeparser()
) main;
```

2.4 BMv2-Specific Notes

> [!IMPORTANT]
> **BMv2 simple_switch** provides these features used by P4air:
>
> | Feature | BMv2 Support | How |
> |---|---|---|
> | Priority Queues | ✅ | `--priority-queues 8` flag at startup |
> | `standard_metadata.enq_qdepth` | ✅ | Available in egress block |
> | `standard_metadata.ingress_global_timestamp` | ✅ | 48-bit microsecond timestamp |
> | `recirculate()` | ✅ | `recirculate(meta)` in egress |
> | `mark_to_drop()` | ✅ | Already used in your KBCS code |
> | Hash computation | ✅ | CRC16/CRC32 supported |
> | Registers (stateful) | ✅ | Already used in your KBCS code |
>
> **NOT available** on BMv2 (paper's Tofino-specific):
>
> - Line-rate processing → Not needed, BMv2 is for functional correctness
> - Multi-pipe parallelism → Single pipeline in BMv2
> - Exact SRAM/TCAM allocation → BMv2 uses host memory

2.5 Tasks — Phase 2

- [ ] Create `headers.p4` (reuse Ethernet/IPv4/TCP from KBCS, add P4air metadata)
- [ ] Create `parser.p4` (same structure as KBCS parser)
- [ ] Create `p4air.p4` top-level with empty ingress/egress
- [ ] Add all register declarations
- [ ] Add basic `ipv4_lpm` forwarding table (copy pattern from KBCS)
- [ ] Compile with `p4c-bm2-ss --p4v 16 -o build/p4air.json p4src/p4air.p4`
- [ ] Run basic forwarding test with Mininet (ping between hosts)

Phase 3 — Fingerprinting Module

3.1 Overview

The fingerprinting module is the **core** of P4air. It classifies each TCP flow into one of 4 CCA groups by observing packet patterns over RTT-length time windows.

3.2 Flow Lifecycle

```
New flow → ANT (if non-TCP/very few pkts)
         → MICE (TCP, in slow-start)
            ↓ (slow-start ends)
           DELAY-BASED (default for all long-lived flows)
            ↓ (queue filling detected for mLD RTTs)
           LOSS-DELAY
            ↓ (continuous filling for mPL RTTs)
           PURELY LOSS-BASED

           At any point, periodic BW probing pattern → MODEL-BASED (BBR)
```

3.3 RTT Estimation (Ingress)

```
1. On SYN packet (tcp.flags & 0x02):
   - Store timestamp: reg_syn_ts[flow_id] = ingress_global_timestamp
   - Set flag: reg_syn_seen[flow_id] = 1

2. On first data packet after SYN:
   - RTT = ingress_global_timestamp - reg_syn_ts[flow_id]
   - Store: reg_rtt[flow_id] = RTT
```

```
  - Initialize RTT interval: reg_rtt_start[flow_id] =
ingress_global_timestamp
```

3.4 BDP Calculation (Hardware Approximation)

```
BDP ≈ RTT_estimate >> s
where s = ⌈log₂(num_flows) + log₂(pkt_len × throughput)⌉
```

On BMv2, we can use division since it's software, but keep the shift for consistency:

```
// Simple approximation for BMv2
meta.bdp = (bit<32>)(meta.rtt_estimate >> 4);  // Adjust shift based on s
```

3.5 Slow-Start End Detection

Transition from MICE → DELAY-BASED when:

1. `num_pkts_current < num_pkts_prev` (sending rate decreased), **OR**
2. `num_pkts_current ≥ BDP` (reached fair share → **proactively DROP 1 packet**)

3.6 Per-RTT Statistics (Ingress)

Every packet:

```
reg_num_pkts.read(meta.num_pkts, flow_id);
meta.num_pkts = meta.num_pkts + 1;
reg_num_pkts.write(flow_id, meta.num_pkts);
```

At end of RTT interval (when `current_time - rtt_start > rtt_estimate`):

```
1. Update BwEst: if num_pkts ≥ num_pkts_prev + (num_pkts_prev >> 3) →
bwest++
2. Save current → previous
3. Reset current to 0
4. Update rtt_start = current_time
```

3.7 Per-RTT Statistics (Egress)

Every packet:

```
// Read enqueue depth from standard_metadata
if (standard_metadata.enq_qdepth > meta.max_enq_len) {
    meta.max_enq_len = standard_metadata.enq_qdepth;
}
```

At end of RTT interval:

```
1. Update aggressiveness:
   if max_enq > max_enq_prev + (max_enq_prev / 100) → aggr_streak++
   else → aggr_streak = 0

2. Save current → previous
3. Reset current to 0
```

3.8 Group Recalculation Rules

Transition	Condition	Default Threshold
delay → loss-delay	`aggr_streak ≥ mLD` and no decrease in `num_pkts`	`mLD = 4`
loss-delay → purely-loss	`aggr_streak ≥ mPL` (continuous queue filling)	`mPL = 12`
any → model-based	Periodic increase/decrease in sending rate, `bwest ≥ mM`	`mM = 4`

3.9 Recirculation on Group Change

When egress detects a group change:

```
// In egress: if group changed
if (meta.group_changed == 1) {
    // Recirculate packet so ingress can update the group register
    recirculate(meta);
}
```

3.10 Tasks — Phase 3

- [ ] Implement 5-tuple hash for `flow_id` (reuse KBCS pattern)
- [ ] Implement SYN detection and RTT estimation

- ☐ Implement BDP approximation
- ☐ Implement slow-start end detection (both patterns)
- ☐ Implement proactive packet drop at BDP boundary
- ☐ Implement ingress per-RTT stats tracking (`num_pkts` , `bwest`)
- ☐ Implement egress per-RTT stats tracking (`enq_depth` , `aggressiveness`)
- ☐ Implement delay → loss-delay reclassification
- ☐ Implement loss-delay → purely-loss reclassification
- ☐ Implement model-based (BBR) detection
- ☐ Implement egress recirculation on group change
- ☐ **Test**: Single Cubic flow → should reach "purely loss-based"
- ☐ **Test**: Single Vegas flow → should stay "delay-based"
- ☐ **Test**: Single BBR flow → should reach "model-based"
- ☐ **Test**: Single Illinois flow → should reach "loss-delay"

Phase 4 — Reallocation Module

4.1 Queue Layout for BMv2

```
Start BMv2 with: simple_switch --priority-queues 8

Queue 0: Ants (always)
Queue 1: Mice (always)
Queues 2-7: Dynamically allocated among 4 long-lived groups
  - Initial: Q2-Q3 delay, Q4-Q5 loss-delay, Q6 loss, Q7 model
  - Boundaries shift as flow counts change
```

4.2 Boundary Values (`li`)

```
Group         Queue Range
delay:        [2, l1)
loss-delay:   [l1, l2)
purely-loss:  [l2, l3)
model:        [l3, 7]

Initial: l1=4, l2=6, l3=7
```

4.3 Reallocation Logic

When a recirculated packet arrives (group just changed):

1. Update `reg_group_flows[old_group]--` and `reg_group_flows[new_group]++`
2. Recalculate boundaries: groups with more flows get more queues
3. Assign the flow to a queue within the new group's range using sequential index

For normal packets:

1. Read stored queue from `reg_queue[flow_id]`
2. Check if stored queue is within current group boundaries
3. If outside → reassign

4.4 fpq (Flows Per Queue) Calculation

```
fpq = total_long_lived_flows / num_dynamic_queues
    = total_long_lived_flows / 6       (queues 2-7)
```

4.5 Tasks — Phase 4

- [ ] Implement group boundary registers (`l1, l2, l3`)
- [ ] Implement per-group flow counter updates
- [ ] Implement boundary recalculation on reclassification
- [ ] Implement queue assignment (sequential index per group)
- [ ] Implement queue boundary check for already-assigned flows
- [ ] **Test**: Start with 4 flows (one per group) → verify each gets separate queue
- [ ] **Test**: Add many loss-based flows → verify loss group gets more queues

---

Phase 5 — Apply Actions Module

5.1 Trigger Condition

Actions applied **only when the flow exceeds its fair share**:

```
if (meta.num_pkts > meta.bdp) {
    // Apply group-specific action
}
```

5.2 Per-Group Actions

Group	Action	P4 Implementation	Why

Group	Action	P4 Implementation	Why
Purely loss-based	Drop packet	`mark_to_drop()`	Uses loss as primary metric
Loss-delay	Drop packet	`mark_to_drop()`	Also reacts to loss
Delay-based	Delay packet	`recirculate(meta)`	Reacts to RTT increase
Model-based (BBR)	Reduce window	`hdr.tcp.window = hdr.tcp.window >> 1`	Doesn't react to loss/delay

5.3 Window Modification (Model-Based)

```
action adjust_receiver_window() {
    // Halve the receiver window in ACK packets going back to sender
    hdr.tcp.window = hdr.tcp.window >> 1;
    // Note: TCP checksum must be recomputed in ComputeChecksum block
}
```

> [!WARNING]
> **Requirement**: For window modification to work, both directions of the flow (data and ACKs) must traverse the same P4air switch. This is naturally the case in the dumbbell topology where all traffic goes through the single bottleneck switch.

5.4 Sensitivity Parameter (`s`)

- **Too high** → over-punishment → low utilization (~50%)
- **Too low** → under-punishment → unfair
- **Sweet spot** → ~90% utilization with high fairness

$$s = \lceil \log_2(num_flows) + \log_2(pkt_len \times throughput) \rceil$$

5.5 Tasks — Phase 5

- [ ] Implement BDP-based trigger condition in ingress
- [ ] Implement `drop_packet` for loss-based and loss-delay groups
- [ ] Implement `delay_packet` (recirculation) for delay-based group
- [ ] Implement `adjust_window` for model-based group
- [ ] Recalculate TCP checksum after window modification
- [ ] **Test**: Cubic flows → packets dropped when exceeding BDP → CWND drops
- [ ] **Test**: Vegas flows → packets delayed → RTT increases → backs off

- [ ] **Test**: BBR flows → window reduced → sending rate decreases
- [ ] Tune sensitivity `s` for ≥90% utilization

Phase 6 — Evaluation & Comparison

6.1 Topology (BMv2 + Mininet Only)

```
                                        ┌── S1
C1  (CCA₁) ──┐                          │
C2  (CCA₂) ──┤        ┌──────────┐      ├── S2
   ...       ├────────┤  P4air   ├──────┤
Cn  (CCAₙ) ──┘        │  Switch  │      ├── ...
                      │  (BMv2)  │      │
                      └──────────┘      └── Sn
```

```
Bottleneck link (to servers): 10 Mbps, configurable delay & queue size
Client links: 100 Mbps (non-bottleneck)
RTTs: Configured via Mininet TCLink delay parameter
Queue size: 100 packets (MAX_QUEUE_SIZE)
```

> [!NOTE]
> This is the **same topology pattern** as your KBCS setup (`topology.py`),
> just generalized for N clients with configurable CCAs and RTTs.
> Paper's BMv2 topology (Fig. 5a) used 1000 pps output rate;
> in Mininet we use bandwidth limiting via `TCLink(bw=10)` for comparable behavior.

6.2 Four Configurations to Compare

Config	Description	Implementation
No AQM	Simple forwarding, no fairness	Just `ipv4_lpm` table, FIFO queue
Different Queues	Hash-based queue separation (vendor-style)	Hash 5-tuple → select queue
Idle P4air	Fingerprinting + Reallocation only	P4air without Apply Actions
P4air	Full solution	All three modules

6.3 Experiments

Experiment 1: Fingerprinting Accuracy

```
Setup:  1 flow at a time, each of the 4 representative CCAs
        Cubic, Illinois, Vegas, BBR
Config: Queue=100 pkts, Bottleneck=10Mbps, RTT=100ms
Vary:   mLD (1-6), mPL (6-18), mM (1-6)
Measure: Detection accuracy (%) and delay (RTT intervals)
Runs:   10 per CCA × 10 RTT values (50-150ms, step 10)
```

Experiment 2: Inter/Intra Fairness

```
Setup:  4-128 flows, mix of CCA groups
Config: Group mix varied 25% steps = 35 combos
Measure: Jain's Fairness Index, utilization
Runs:   4 per combination
Tool:   iperf3 (parallel flows to server)
```

Experiment 3: RTT Fairness

```
Setup:  Multiple flows, same CCA, different RTTs
Config: ΔRTT = 0-10ms (via Mininet link delay)
Measure: Jain's Fairness Index per CCA
Runs:   4 per scenario
```

Experiment 4: Comparison (All Baselines)

```
Setup:  Representative scenario (e.g., 8 flows: 2 Cubic + 2 Illinois + 2
Vegas + 2 BBR)
Run all 4 configs: No AQM, Diff Queues, Idle P4air, P4air
Compare: Fairness Index, Utilization, RTT increase
```

6.4 Evaluation Metrics

Metric	Formula
Jain's Fairness Index	$J = (\Sigma x_i)^2 / (n \times \Sigma x_i^2)$ where x_i = flow throughput
Utilization	$U = \Sigma(\text{throughput}) / \text{link_capacity} \times 100$
Detection Delay	RTT intervals until correct group assignment
Detection Accuracy	correct / total × 100%

6.5 Scaling Down for VM

> [!IMPORTANT]
> The paper tests up to **256 flows** on hardware. On a VM, scale down:
>
> - Use **4–16 flows** maximum for reliable BMv2 emulation
> - Use **10 Mbps** bottleneck (not 10 Gbps — impossible in VM)
> - Keep shorter test durations (30-60s instead of minutes)
> - Results will show **same trends** (fairness improvement) at smaller scale

6.6 Tasks — Phase 6

- [ ] Build topology.py for P4air (generalize from KBCS's topology.py)
- [ ] Implement "No AQM" baseline (like kbcs_baseline.p4 — just forwarding)
- [ ] Implement "Different Queues" baseline (add hash-based queue selection)
- [ ] Implement "Idle P4air" variant (disable Apply Actions)
- [ ] Write `run_fingerprinting_test.py` script
- [ ] Write `run_fairness_test.py` script
- [ ] Write `calculate_fairness.py` (Jain's Index from iperf3 JSON)
- [ ] Write `plot_results.py` (matplotlib charts)
- [ ] Run Experiment 1 (fingerprinting accuracy)
- [ ] Run Experiment 2 (inter/intra fairness)
- [ ] Run Experiment 3 (RTT fairness)
- [ ] Run Experiment 4 (full comparison)
- [ ] Generate summary comparison table & charts

Key Equations

Jain's Fairness Index

```
J(x₁,...,xₙ) = (Σxᵢ)² / (n × Σxᵢ²)        Range: [1/n, 1]
```

RTT Estimation

```
RTT = timestamp(first_data_after_SYN) - timestamp(SYN)
```

BDP (Bandwidth-Delay Product)

```
BDP = (output_rate / num_flows) × RTT
Hardware approximation: BDP ≈ RTT >> s
  s = ⌈log₂(n_flows) + log₂(pkt_len × throughput)⌉
```

Aggressiveness

```
if max_enq_current > max_enq_prev × 1.01 → aggr_streak++
else → aggr_streak = 0
```

BwEst (Bandwidth Estimation)

```
if num_pkts_current ≥ num_pkts_prev × 1.125 → bwest++
Approximation: num_pkts ≥ num_pkts_prev + (num_pkts_prev >> 3)
```

Parameter Reference

Parameter	Default	Description
`mLD`	4	RTT intervals for delay→loss-delay
`mPL`	12	RTT intervals for loss-delay→loss
`mM`	4	BW probing patterns for BBR detection
`s`	Calculated	Apply Actions sensitivity
`FLOW_TABLE_SIZE`	1024	Max flows tracked
`NUM_QUEUES`	8	Total queues (BMv2 `--priority-queues 8`)
`MAX_QUEUE_SIZE`	100	Max pkts per queue
Bottleneck BW	10 Mbps	Mininet `TCLink(bw=10)`
Client BW	100 Mbps	Mininet `TCLink(bw=100)`
Link delay	5-50ms	Mininet `TCLink(delay='Xms')`

Phase Dependencies

```
Phase 1 (Setup) → Phase 2 (Pipeline) → Phase 3 (Fingerprinting)
                                              ↓
                                  ┌──────────┴──────────┐
                                  ↓                     ↓
                         Phase 4 (Reallocation)   Phase 5 (Apply Actions)
                                  ↓                     ↓
                                  └──────────┬──────────┘
```

```
                    ↓
Phase 6 (Evaluation)
```

> **Phase 4 and Phase 5 can run in parallel** after Phase 3.